

**SURPLUS MARKETPLACE – SMART SUSTAINABLE ECOMMERCE
PLATFORM
SURPLUS LITE – CLOUD-BASED MULTI-VENDOR MARKETPLACE**

CLOUD COMPUTING

Submitted by

SHERYL KATRINA M (230701310)

SANDHIYA (230701282)

SAKSHI P (230701279)

in partial fulfilment for the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "SURPLUS MARKETPLACE - A SMART SUSTAINABLE E-COMMERCE PLATFORM" is the bonafide work of SHERYL KATRINA M (230701310), SANDHIYA P (230701282), SAKSHI P (230701279) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor & Supervisor
Department of Computer Science
Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. M. Ayyadurai
Head of The Department Associate Professor
Department of Computer Science and
Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner
External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S.MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr.MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, DR. M. AYYADURAI M.E.,Ph.D., Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

SHERYL KATRINA M
SANDHIYA P
SAKSHI P

ABSTRACT

The rapid growth of retail and food industries has led to a significant increase in surplus and unsold products, contributing to economic losses and environmental waste. Vendors often face challenges in disposing of excess inventory before expiration, while buyers lack awareness of nearby discounted deals. This gap leads to inefficiencies in resource utilization and missed economic opportunities.

This project proposes Surplus Lite, a smart sustainable e-commerce platform designed to connect vendors and buyers in real time through a location-aware system. Vendors can quickly upload surplus products, while buyers can discover nearby deals based on proximity and price. The platform integrates modern technologies such as React for frontend development, Node.js and Express for backend services, MongoDB for data storage, and AWS for cloud infrastructure.

Intelligent features such as recommendation systems, chatbot assistance, and smart sorting enhance user experience and decision-making. The system aims to reduce product waste, increase vendor revenue recovery, and provide affordable options to consumers. By promoting sustainable consumption and efficient resource utilization, the platform contributes to both economic and environmental benefits.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	Introduction	1
	1.1 Problem Statement	2
	1.2 Objective of the Project	4
	1.3 Scope and Boundaries	6
	1.4 Stakeholders and End Users	8
	1.5 Technologies Used	10
	1.6 Organization of the Report	12
2	System Design and Architecture	14
	2.1 Requirement Summary (Functional & Non-Functional)	15
	2.2 Proposed Solution Overview	17
	2.3 Cloud Deployment Strategy	19
	2.4 Infrastructure Requirements	21
	2.5 AWS Services Mapping and Justification	23
3	Implementation	26
	3.1 Continuous Integration and Development	27
	3.2 Cloud Deployment Workflow	29
	3.3 Deployment Versioning and Release Management	31
	3.4 Resource Scaling and Performance Management	33
	3.5 Gen AI Integration and AWS AI Service Mapping	35
4	Cloud Operations and Security	38
	4.1 DevSecOps Integration	39
	4.2 Monitoring and Observability	41
	4.3 Access Control	43
	4.4 Performance Optimization	45
	4.5 Reliability and Availability	45
5	Results and Discussion	48
	5.1 Implementation Summary	49
	5.2 Challenges Faced and Resolutions	51

	5.3 Performance or Cost Observations	53
	5.4 Key Learnings and Team Contributions	55

	Conclusion and Future Enhancement	58
	6.1 Conclusion	59
	6.2 Future works	61
	References	63
	Appendices	65

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

In today's retail and food industries, a significant amount of products remain unsold and eventually go to waste. This issue is especially critical for vendors dealing with perishable or time-sensitive goods, where unsold inventory directly results in financial losses. Studies and real-world observations indicate that a large percentage of products expire before they can be sold, contributing not only to economic inefficiency but also to environmental damage.

One of the primary reasons for this problem is the lack of an efficient platform that enables vendors to quickly offload surplus inventory. Traditional sales channels are often slow and not designed for urgent clearance of excess stock. As a result, vendors are unable to recover the value of unsold goods.

On the consumer side, buyers are constantly looking for affordable products and discounts, but they often lack awareness of available deals in their local area. Although surplus products exist in large quantities, there is no effective mechanism to connect these products with nearby buyers in real time. This gap between supply and demand leads to missed opportunities for both vendors and consumers.

Existing e-commerce platforms are not optimized for handling surplus or expiring products. They primarily focus on standard retail transactions and lack features such as location-based discovery, real-time updates, and intelligent recommendations tailored for surplus goods. Additionally, these platforms do not prioritize sustainability or waste reduction.

The absence of a dedicated system for surplus product management results in:

- Increased product waste and environmental impact
- Financial losses for vendors
- Limited access to affordable products for buyers
- Inefficient utilization of available resources

Therefore, there is a need for a smart, efficient, and scalable platform that connects vendors and buyers, enabling real-time transactions of surplus products while promoting sustainability and economic efficiency.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and develop a smart sustainable e-commerce platform called Surplus Lite, which facilitates efficient buying and selling of surplus products through a location-aware system. The project aims to address the challenges of product waste, vendor losses, and lack of deal discovery by implementing a platform that connects vendors with nearby buyers in real time.

The specific objectives of the project are as follows:

Reduce Product Waste: Enable vendors to quickly list and sell surplus or expiring products, thereby minimizing waste and promoting sustainable consumption.

Improve Vendor Revenue Recovery: Provide vendors with an efficient platform to recover financial losses by selling excess inventory at discounted prices.

Enhance Buyer Accessibility: Allow buyers to discover affordable products in their local area through location-based filtering and real-time updates.

Implement Smart Recommendation System: Use intelligent algorithms to suggest relevant products based on user preferences, browsing history, and price similarity.

Provide User-Friendly Experience: Design an intuitive interface that simplifies product browsing, comparison, and purchasing for both vendors and buyers.

Ensure Scalable Cloud Deployment: Utilize cloud technologies to build a scalable and reliable system capable of handling multiple users and vendors simultaneously.

Promote Sustainable Practice: Encourage efficient utilization of resources by reducing waste and supporting local businesses.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the development and deployment of a full-stack web application designed to facilitate efficient surplus product transactions between vendors and buyers. The system integrates cloud infrastructure and intelligent features to ensure scalability, performance, and usability. It includes functionalities such as user authentication, product management, location-based discovery, recommendation systems, and chatbot-assisted interaction to provide a seamless marketplace experience.

The platform supports core marketplace operations including user registration, vendor-based product listing, browsing and filtering of products, and cart and order management. Vendors can upload surplus inventory with relevant details, while buyers can discover nearby deals based on proximity and pricing. The system also incorporates a recommendation mechanism to provide personalized suggestions and a rule-based chatbot to assist users in navigation and queries.

The backend is developed using Node.js and Express, while the frontend is built using React. MongoDB is used for data storage, and the system is deployed on AWS with Cognito handling authentication. The application is currently limited to a web-based platform, with simulated location tracking and no integrated payment gateway or advanced AI models, but it provides a strong foundation for future enhancements.

1.4 STAKEHOLDERS AND END USERS

Stakeholders:

1. **Project Developers:** Responsible for designing, developing, and maintaining the system components including frontend, backend, and cloud infrastructure.
2. **Vendors Sellers:** Businesses or individuals who list surplus products on the platform to sell them before expiration.
3. **Platform Administrators:** Responsible for managing system operations, monitoring performance, and ensuring smooth functioning of the platform.

End Users

1. **Buyers Consumers:** Individuals looking for affordable products and local deals. They use the platform to browse, compare, and purchase surplus items.
2. **Local Businesses:** Small-scale vendors who benefit from increased visibility and the ability to sell excess inventory efficiently.
3. **Eco-Conscious Users:** Users who prefer sustainable consumption and aim to reduce waste by purchasing surplus products.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

React	For building frontend user interface of marketplace
Local Storage	Used for managing session data and improving user experience.

Table 2: Backend Technologies

Node.js	Provides a runtime environment for executing server-side logic.
Express.js	Used to build REST APIs for handling requests and responses efficiently.
Mongodb	A NoSQL database used to store user data, product listings, and transaction details.

Table 3: Cloud and Infrastructure

AWS(Amazon Web Server)	Used for hosting and managing cloud services.
AWS Cognito	Provides secure user authentication and identity management.

Table 4: AI and Machine Learning

Recommendation System	Suggests products based on user behavior and preferences.
Rule based chatbot	Assists users in navigating the platform and resolving queries.

Table 5: DevOps and CI/CD

GitHub Actions	For automating workflows, builds, and deployments.
Docker Hub	As a registry for storing and distributing container images.
Azure CLI	For command-line management of Azure resources.

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, solution overview, cloud strategy, infrastructure specifications, and service mappings.

Chapter 3 delves into the DevOps implementation, covering CI/CD pipelines, Terraform automation, Docker containerization, orchestration principles, and AI service integration.

Chapter 4 focuses on operational excellence and security, discussing DevSecOps practices, monitoring, access control, and deployment strategies.

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, cost analysis, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing configuration details and system screenshots.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

1.1 REQUIREMENT SUMMARY Functional Requirements

The proposed system is a cloud-based multi-vendor marketplace designed to enable seamless interaction between vendors and buyers for surplus product transactions. The frontend of the application is developed using React, providing an intuitive and responsive user interface that allows users to browse products, view vendor details, and perform transactions efficiently.

The backend is implemented using Node.js and Express, which handle core functionalities such as user authentication, product listing, recommendation processing, and order management. REST APIs are used to facilitate communication between the frontend and backend components, ensuring smooth data exchange and system responsiveness. MongoDB is used as the primary database to store user information, product details, and transaction data, providing flexibility and scalability for dynamic content.

The system supports vendor registration and product listing, enabling sellers to upload surplus inventory with relevant details. Buyers can register and browse available products, apply filters based on their preferences, and discover items using location-based search. The platform also includes a recommendation system that suggests products based on user behavior and preferences, along with a chatbot that assists users in navigation and query handling. Additionally, cart and order management functionalities are provided to ensure a complete transaction workflow from product selection to purchase.

The cloud infrastructure is supported by AWS services, where Cognito is used for authentication and cloud hosting ensures reliable deployment and accessibility of the application.

Non-Functional Requirements

The system is designed to meet various non-functional requirements to ensure performance, reliability, and scalability. The platform aims to provide fast response times with minimal latency, ensuring that product listings and recommendations are delivered efficiently to users. It is designed to support multiple users and vendors simultaneously, with cloud-based deployment enabling scalability as demand increases.

High availability is maintained to ensure uninterrupted access to the system, minimizing downtime and providing a consistent user experience. Security is enforced through secure authentication using AWS Cognito, along with protected API communication and safe data storage practices. The user interface is designed to be simple and intuitive, ensuring ease of use for both vendors and buyers.

The system architecture is modular, allowing easy maintenance and future enhancements without affecting existing functionalities. This ensures long-term sustainability and adaptability of the platform as new features and improvements are introduced.

1.2 PROPOSED SYSTEM SOLUTIONS

The proposed solution is a smart sustainable e-commerce platform named **Surplus Lite**, which focuses on reducing product waste by connecting vendors with buyers in real time.

The system is designed with a layered architecture consisting of a frontend, backend, database, and cloud services. The frontend provides an interactive interface for users, while the backend handles business logic and communication with the database.

Vendors can upload surplus products quickly, including details such as price, category, and availability. Buyers can view nearby products using location-based filtering and select the best deals based on price and distance.

The system integrates intelligent features such as recommendation engines that analyze user behavior and suggest relevant products. A rule-based chatbot assists users in navigating the platform and resolving queries.

Overall, the solution aims to improve efficiency in product distribution, reduce waste, and provide economic benefits to both vendors and buyers.

1.3 CLOUD DEPLOYMENT STATERGY

The deployment strategy of the proposed system adopts a cloud-native approach to ensure scalability, reliability, and efficient resource utilization. The application is deployed using Amazon Web Services (AWS), which provides a flexible and secure environment for hosting both frontend and backend components. This approach enables the system to handle varying user loads while maintaining consistent performance.

The frontend application, developed using React, is hosted on cloud services, allowing users to access the platform through a web browser with minimal latency. The backend services, built using Node.js and Express, are deployed as RESTful APIs that process user requests, manage business logic, and interact with the database. These services are designed to be scalable and capable of handling concurrent user interactions efficiently.

User authentication and identity management are handled using AWS Cognito, which ensures secure access control and protects user data. MongoDB is used as the primary database for storing user information, product listings, and transaction data, providing flexibility and scalability for dynamic operations. The communication between frontend and backend is managed through secure API calls, ensuring data integrity and system reliability.

The overall deployment architecture supports continuous availability, efficient data handling, and seamless integration of system components. This cloud-based strategy ensures that the platform can scale with increasing demand while maintaining performance and security standards, making it suitable for real-time marketplace operations.

1.4 INFRASTRUCTURE REQUIREMENTS

ComputeResources:• **AWS EC2 / Elastic Beanstalk:** Instances used to host the Node.js and Express backend services, supporting scalable deployment and handling multiple user requests efficiently.• **AWS Cognito:** Used for managing user authentication and identity services, ensuring secure access control.

StorageRequirements:• **MongoDB Atlas:** Cloud-hosted NoSQL database used for storing user data, product listings, and transaction details with high scalability and flexibility.• **Amazon S3:** Used for storing product images and static frontend assets, ensuring reliable and cost-effective storage.

1.5 AWS SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	Azure Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost (₹)
Frontend Hosting	Amazon S3 + CloudFront	Static Hosting + CDN	Host React UI	Fast delivery, low latency, scalable	Free (Student Credit)

Backend Logic & APIs	AWS EC2 / Elastic Beanstalk	t2.micro instance	Run Node.js backend	Scalable and reliable compute	2000
Database	MongoDB Atlas	Shared Cluster	Store user and product data	Flexible NoSQL storage	1,200
Media Storage	Amazon S3	Standard Storage	Store product images	Cost Efficient	300
Authentication	AWS Cognito	Standard	Manage user login	Secure and scalable identity management	Free
Recommendation System	Custom ML Logic	Basic Implementation	Suggest products	Improves user experience	Free
Chatbot	Rule-Based System	Integrated Backend	Assist users	Simple and effective guidance	Free

CI/CD Automation	Github Actions	Basic	Automated builds & deployments	Seamless GitHub integration	Free (Student Credit)
-------------------------	----------------	-------	--------------------------------	-----------------------------	-----------------------

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses a multi-stage CI/CD pipeline built with GitHub Actions to automate the deployment process. This setup integrates cloud deployment, version control, and automation to ensure efficient delivery of application updates. The pipeline is designed to handle both frontend and backend deployment in a structured and reliable manner.

The pipeline follows 2-stage deployment pattern:

1. **Backend:** Deploys the Node.js application on AWS services.
2. **Frontend:** Builds and deploys the React application as a static web application.

The entire process is triggered either by a push to the main branch or through a manual workflow dispatch.

Pipeline Stages in Detail:

Stage 1: Backend Deployment (build-and-deploy-backend)

1. **Versioning and Build:** A unique version tag is generated using the date and GitHub commit ID (e.g., v20260504-a1b2c3d). The backend application is built and prepared for deployment.
2. **Deployment:** The backend is deployed on AWS EC2 or Elastic Beanstalk, where the Node.js application runs and handles API requests.
3. **Secure Access:** Authentication credentials and configuration values are managed securely using GitHub Secrets.
4. **Health Check:** After deployment, the pipeline performs validation checks to ensure that the backend service is running correctly before proceeding.

Stage 2: Frontend Deployment (build-and-deploy-frontend)

Dynamic Configuration: The backend API URL is passed dynamically to the frontend build process to ensure proper communication between components.

Hosting: The optimized React build is deployed to Amazon S3 and distributed using CloudFront for fast content delivery.

3.2 CLOUD DEPLOYMENT WORKFLOW

The project follows a structured cloud deployment workflow using AWS services and GitHub Actions to ensure reliable application deployment and management.

Frontend Deployment

The React frontend application is built and deployed as a static web application. The optimized build files are hosted using Amazon S3 and distributed through CloudFront for faster content delivery and reduced latency.

Backend Deployment

The Node.js and Express backend application is deployed using AWS Elastic Beanstalk. The backend handles API requests, user authentication, product management, and recommendation functionalities.

Database Connectivity

MongoDB Atlas is used as the cloud-hosted database service. The backend securely connects to MongoDB Atlas using environment variables and protected credentials.

Deployment Automation

GitHub Actions is used to automate build and deployment tasks. Whenever changes are pushed to the repository, the workflow automatically triggers the deployment process for frontend and backend components.

Environment Management

Sensitive configuration values such as API keys, database URLs, and authentication credentials are securely managed using GitHub Secrets and AWS configuration settings.

3.3 DEPLOYMENT VERSIONING AND RELEASE MANAGEMENT

The project follows a structured release management approach to ensure stable and reliable deployments.

Application Versioning

Each deployment is managed using GitHub version control, where updates are tracked through commits and repository history. This allows easy rollback and maintenance of stable application versions.

Release Workflow

The deployment workflow is integrated with GitHub Actions, which automates frontend and backend deployment whenever code changes are pushed to the main branch.

Deployment Validation

Basic health checks are performed after deployment to ensure that backend APIs and frontend services are functioning correctly.

Configuration Management

Environment-specific settings are managed separately using environment variables and cloud configuration settings, ensuring secure and flexible deployments across development and production environments.

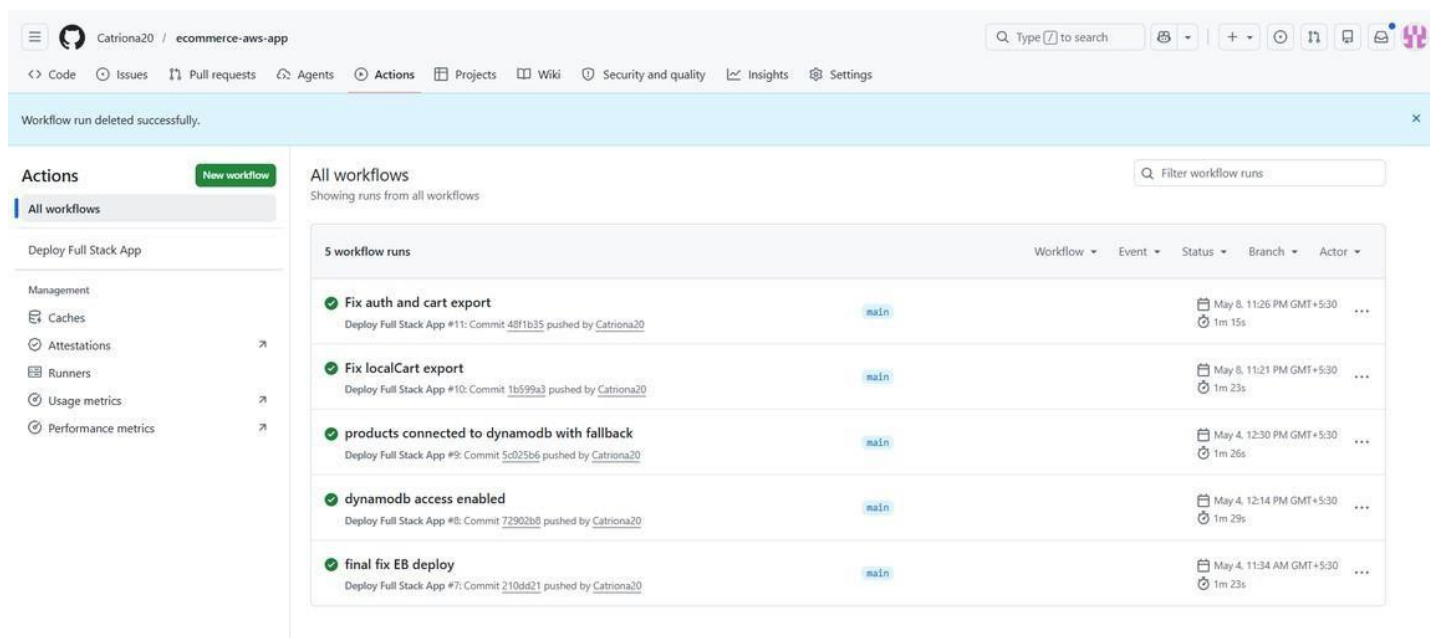


Figure 4. GitHub Actions Deployment Workflow

3.4 RESOURCE SCALING AND PERFORMANCE MANAGEMENT

AWS Elastic Beanstalk is utilised for application deployment and resource management, providing control over scaling, performance, and traffic distribution.

1. Cluster and Deployment Design

Cluster Structure: The backend application is deployed using Elastic Beanstalk, which internally manages EC2 instances, load balancing, and auto-scaling groups. It ensures efficient distribution of resources without requiring manual cluster configuration. **Workload:** The Node.js backend is deployed as an application environment, where multiple instances handle incoming API requests. Health checks are configured to monitor application status through endpoint validation. **Traffic Management:** Traffic is handled using an integrated Elastic Load Balancer, which distributes requests across instances and ensures high availability. HTTPS-based secure communication is enforced for all external access. **Configuration:** Application configuration is managed using environment variables within Elastic Beanstalk. Sensitive data such as credentials is securely managed using AWS services and deployment configurations.

1. Scaling, Resilience, and Security

Instance Scaling: Auto Scaling Groups automatically adjust the number of EC2 instances based on metrics such as CPU utilization and request load. **Resource Optimization:** The system scales down during low traffic and scales up during peak demand, ensuring cost efficiency. **Resilience:** Rolling deployments are supported, ensuring zero downtime during updates and maintaining continuous availability of the application.

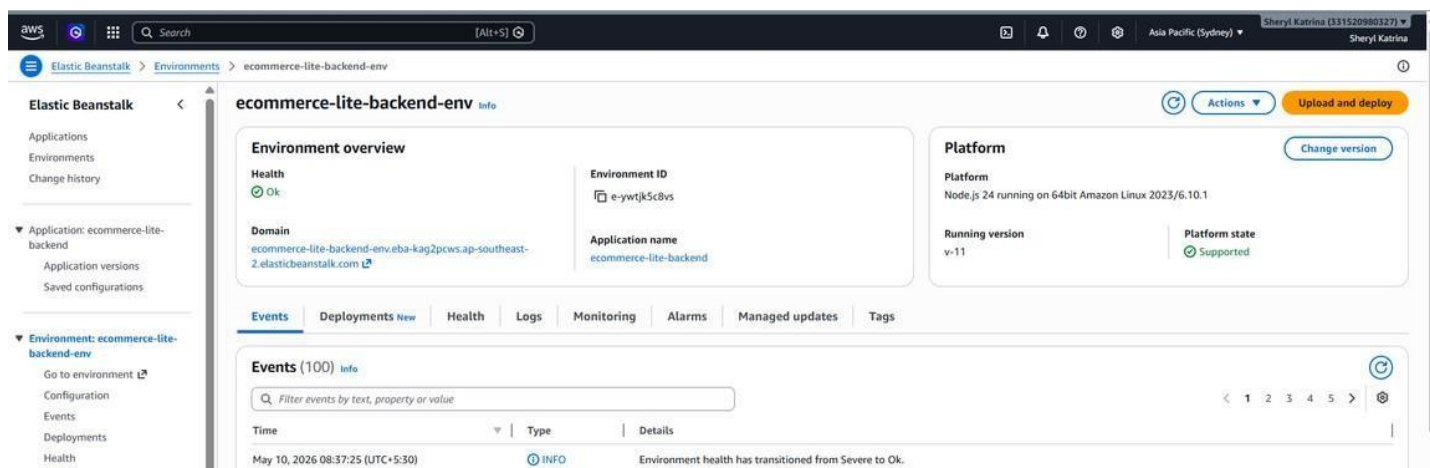


Figure 5. AWS Elastic Beanstalk Environment Dashboard

3.5 GENAI INTEGRATION AND AWSAI SERVICE MAPPING

1. AI-Generated Descriptions (Google Gemini Pro Vision API)

The system incorporates intelligent features to enhance user experience and system usability.

Recommendation Concept

Basic recommendation logic can be implemented to suggest products based on user interactions and browsing patterns, improving engagement and usability.

Implementation: The recommendation logic is integrated within the backend, where user activity is analyzed to provide relevant suggestions.

Process:

1. User browsing behavior is tracked.
2. Similar or popular products are identified.
3. Recommendations are displayed to the user.

2. Security Intelligence

AWS Cognito: Enhances security by providing token-based authentication and access control, ensuring that only authorized users can access protected resources.

Implementation: JWT tokens issued by Cognito are verified by the backend before processing API requests.

Process: User authentication is validated, and secure communication is maintained throughout the system.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1. DEVSECOPS INTEGRATION

1. The system implements monitoring and logging mechanisms to ensure continuous visibility into application performance and system health. AWS CloudWatch is used to collect and monitor metrics related to CPU utilization, request count, latency, and error rates for backend services deployed on Elastic Beanstalk. These metrics help in identifying performance bottlenecks and ensuring optimal resource utilization.
2. Logging is enabled for both frontend and backend components. Backend application logs are captured and stored through Elastic Beanstalk logging features, while access logs can be monitored to track incoming requests and detect anomalies. CloudWatch Logs provides centralized log management, enabling developers to analyze system behavior and troubleshoot issues efficiently.
3. Alerts and notifications can be configured based on predefined thresholds, allowing proactive detection of failures or abnormal activity. This ensures timely response to system issues and helps maintain high availability and reliability.

4.2 MONITORING AND OBSERVABILITY

The system implements monitoring and logging mechanisms to ensure continuous visibility into application performance and system health. AWS CloudWatch is used to collect and monitor metrics related to CPU utilization, request count, latency, and error rates for backend services deployed on Elastic Beanstalk. These metrics help in identifying performance bottlenecks and ensuring optimal resource utilization.

Logging is enabled for both frontend and backend components. Backend application logs are captured and stored through Elastic Beanstalk logging features, while access logs can be monitored to track incoming requests and detect anomalies. CloudWatch Logs provides centralized log management, enabling developers to analyze system behavior and troubleshoot issues efficiently.

Alerts and notifications can be configured based on predefined thresholds, allowing proactive detection of failures or abnormal activity. This ensures timely response to system issues and helps maintain high availability and reliability.

4.3 ACCESS CONTROL

1. Security is a critical aspect of the system, and multiple layers of protection are implemented to safeguard user data and application resources. AWS Cognito is used for authentication and authorization, providing secure user login through OAuth2 and JWT-based mechanisms. Tokens issued by Cognito are verified by the backend before granting access to protected resources.
2. Access control is enforced using role-based permissions, ensuring that users can only perform actions they are authorized for. Sensitive operations such as order placement and data retrieval are protected through secure API endpoints.
3. Data security is maintained through encryption mechanisms. All communication between client and server is secured using HTTPS, ensuring data encryption in transit. Additionally, AWS services provide built-in support for encryption at rest, protecting stored data in DynamoDB and S3.
4. Secure credential management is handled using environment variables and AWS configurations, preventing exposure of sensitive information. This layered security approach ensures confidentiality, integrity, and availability of system data.

4.4 PERFORMANCE OPTIMIZATION

The system is optimized to deliver high performance and low latency using cloud-native techniques. Amazon CloudFront is used as a Content Delivery Network (CDN) to cache static frontend assets and deliver them through edge locations closer to users. This significantly reduces page load time and improves user experience.

Backend performance is optimized through efficient API design and request handling. Elastic Beanstalk ensures load balancing and auto-scaling, which helps maintain consistent performance under varying workloads. DynamoDB provides low-latency data access, ensuring fast retrieval and storage of order information.

Caching strategies are applied where necessary to reduce redundant computations and database queries. These optimizations collectively enhance system responsiveness and ensure smooth operation even during high traffic conditions.

4.5 RELIABILITY AND AVAILABILITY

The system is designed to ensure high reliability and continuous availability using AWS managed services. Elastic Beanstalk automatically distributes application instances across multiple availability zones, reducing the risk of downtime due to infrastructure failures.

Load balancing ensures that incoming requests are evenly distributed across instances, preventing overload on any single server. Auto-scaling mechanisms dynamically adjust resources based on demand, ensuring that the system remains responsive during peak usage.

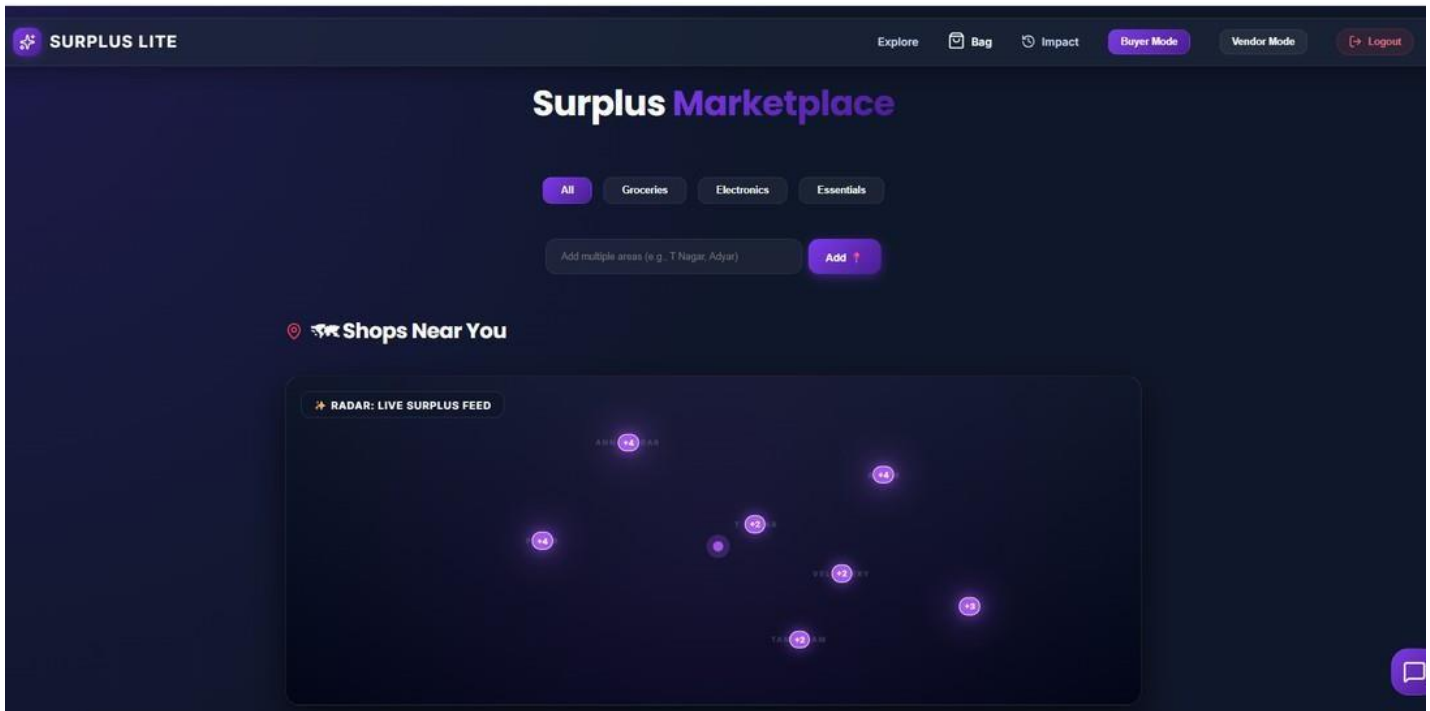
DynamoDB provides built-in replication and fault tolerance, ensuring data durability and availability. CloudFront further enhances reliability by serving cached content even if the origin server experiences temporary issues.

These features collectively ensure that the system remains operational, minimizes downtime, and delivers a consistent user experience.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The proposed cloud-based e-commerce system was successfully implemented and deployed using AWS cloud services. The application provides a complete workflow for product browsing, user authentication, cart management, and order placement through a scalable and secure cloud-native architecture. The frontend application developed using React was hosted on Amazon S3 and delivered through CloudFront, ensuring fast and reliable access for users.



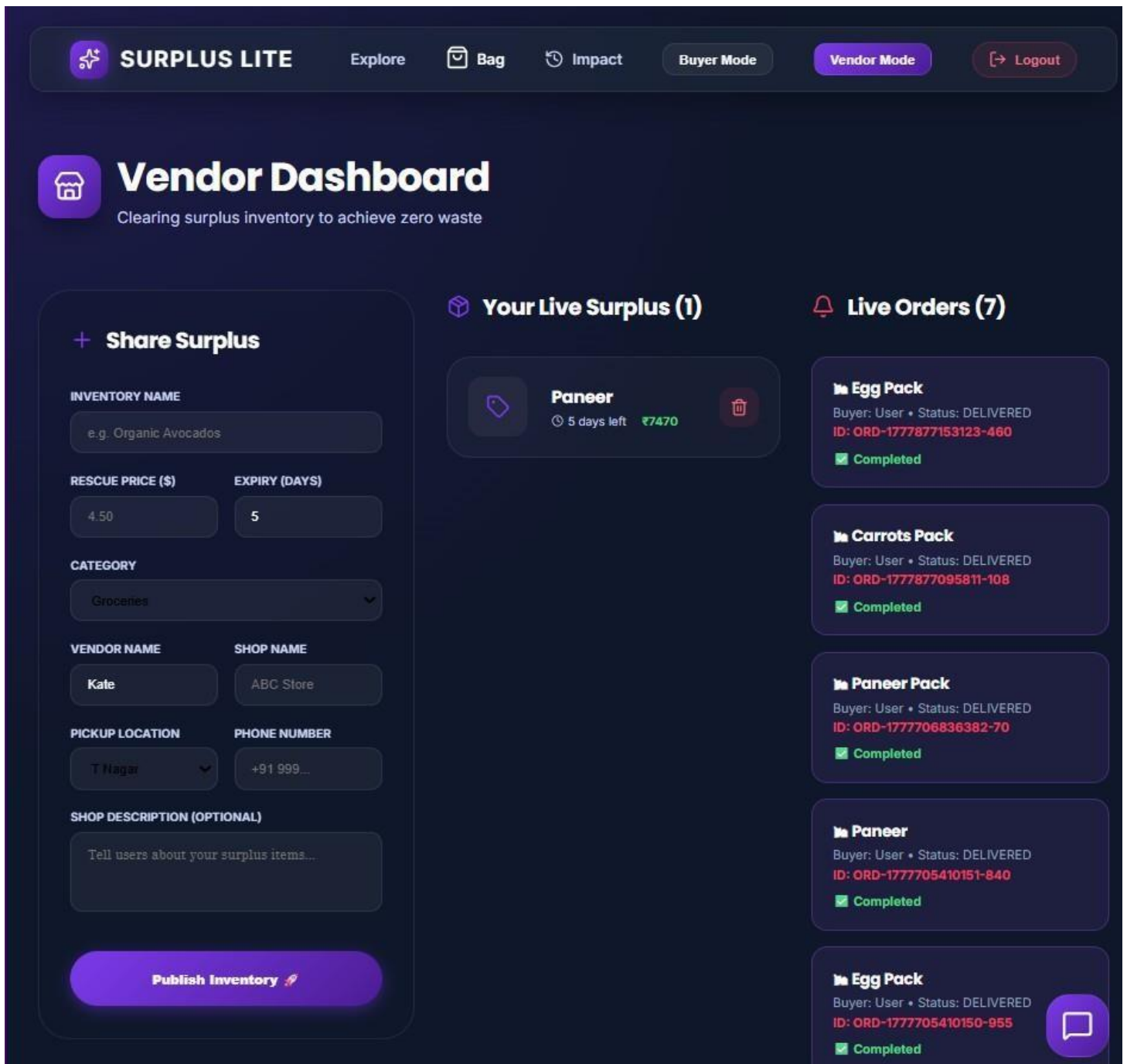


Figure 6. Surplus Lite Homepage Interface

The backend application built using Node.js and Express was deployed using AWS Elastic Beanstalk, which managed the application environment, scaling, and load balancing automatically. AWS Cognito was integrated for secure user authentication and authorization, enabling token-based access control using JWT. DynamoDB was used to store order-related information with low latency and high scalability.



Multi-Vendor History

Tracking your impact across partner stores



SURESH IYER
VEN-1777877153123-SUR

VENDOR TOTAL

\$8.00



Egg Pack
\$8.00



Purchased: 5/4/2026, 12:15:53 PM

⚡ delivered



KARTHIK P
VEN-1777877095811-KAR

VENDOR TOTAL

\$1.00



Carrots Pack
\$1.00



Purchased: 5/4/2026, 12:14:55 PM

⚡ delivered

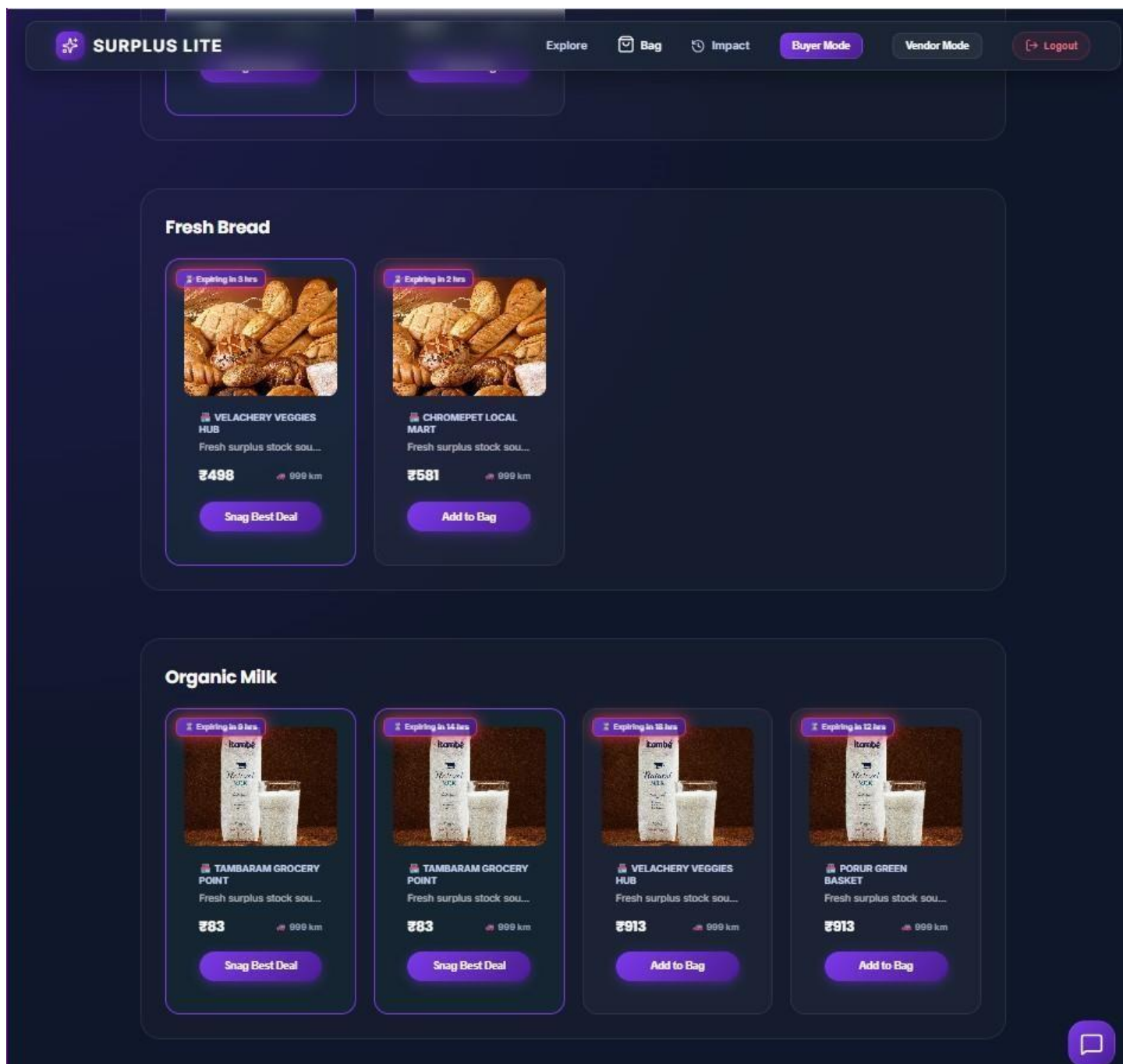


Figure 7. Product Listing and Recommendation Interface

The deployed system successfully demonstrated communication between frontend, backend, authentication, and database services. API requests were processed securely, and user orders were stored and retrieved correctly from DynamoDB. The integration of AWS managed services simplified deployment and improved overall system reliability.

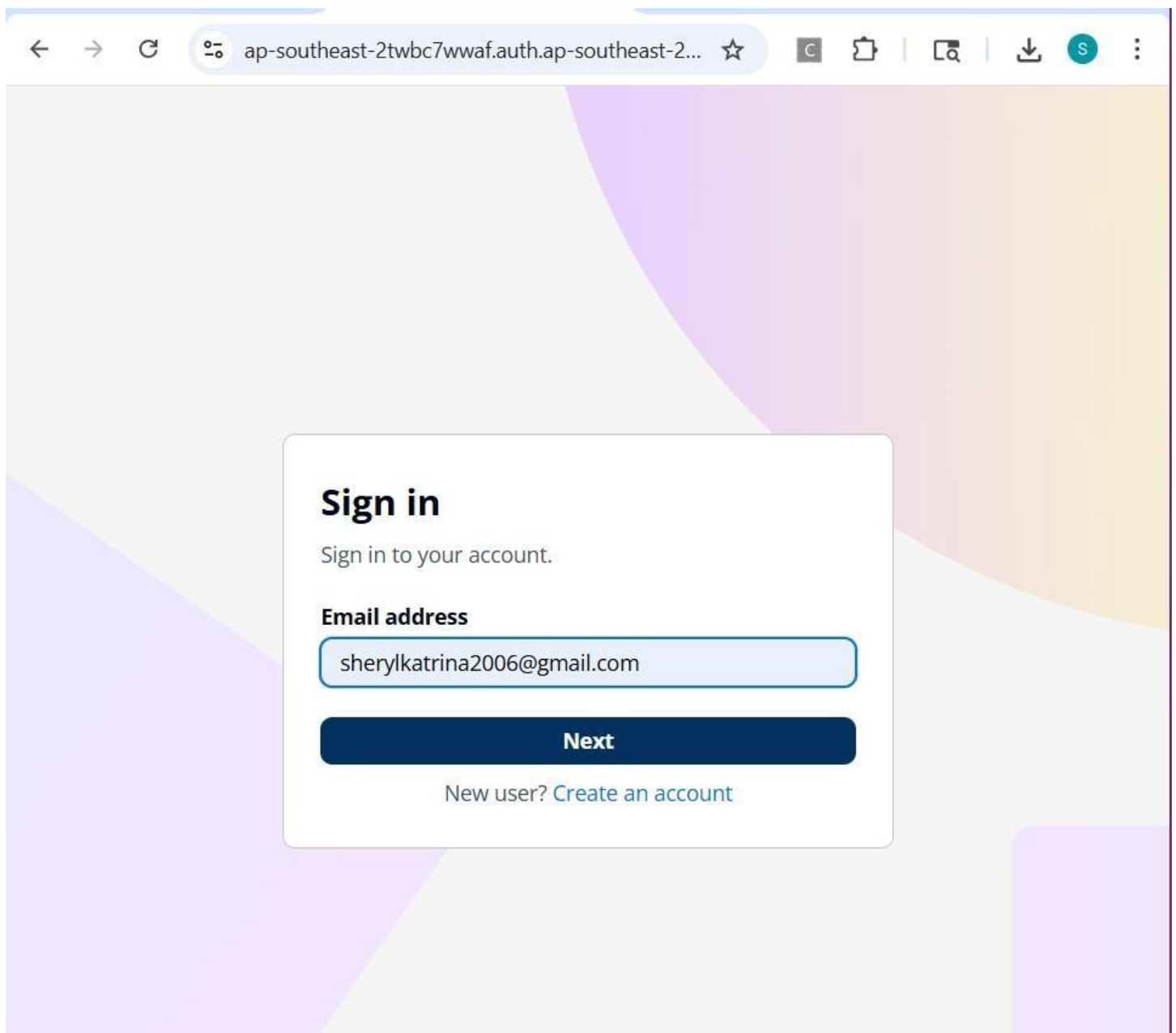


Figure 8. User Authentication Interface

5.2 CHALLENGES FACED AND RESOLUTIONS

Several challenges were encountered during the development and deployment of the system. One of the primary difficulties involved configuring communication between multiple AWS services such as Cognito, Elastic Beanstalk, S3, and DynamoDB. Proper configuration of authentication tokens and API permissions required careful integration and testing.

Another challenge involved deployment automation through GitHub Actions. Managing environment variables, deployment credentials, and workflow configurations required additional setup and debugging. Ensuring correct synchronization between frontend and backend deployments was also an important consideration.

Performance optimization and cloud resource management presented additional challenges, particularly in configuring caching, scaling, and deployment settings efficiently. However, these issues were resolved through testing, monitoring, and proper cloud configuration.

5.3 PERFORMANCE OR COST OBSERVATION Performance Benchmarking

The system achieved good performance during testing and deployment. CloudFront significantly reduced frontend loading time by caching static assets at edge locations, resulting in faster content delivery and lower latency for users. Elastic Beanstalk efficiently handled backend requests and dynamically managed resources based on application demand.

DynamoDB provided quick response times for order storage and retrieval operations due to its optimized NoSQL architecture. The use of managed cloud services reduced infrastructure overhead and ensured stable system operation during continuous usage.

The application demonstrated scalability by supporting multiple concurrent requests while maintaining responsiveness. Auto-scaling and load balancing mechanisms improved availability and prevented server overload during increased traffic conditions.

Cost Analysis and Optimization

The system leverages cloud-native services to achieve cost efficiency while maintaining performance and scalability. AWS follows a pay-as-you-go pricing model, allowing the system to incur costs only for the resources utilized.

Elastic Beanstalk optimizes resource usage through auto-scaling, ensuring that unnecessary compute resources are not consumed during low traffic periods. CloudFront reduces bandwidth costs by caching content and minimizing repeated requests to the origin server.

DynamoDB offers on-demand capacity mode, allowing automatic scaling of read and write operations without manual configuration. This ensures efficient cost management based on actual usage patterns.

By combining scalable services and efficient resource allocation, the system achieves a balance between performance and cost, making it suitable for real-world deployment scenarios.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The project provided practical exposure to cloud-native application development and deployment using AWS services. It helped in understanding concepts such as cloud computing, REST APIs, authentication mechanisms, CDN-based content delivery, and NoSQL database management.

The implementation of CI/CD pipelines using GitHub Actions provided experience in deployment automation and DevOps practices. Working with AWS services such as Cognito, S3, CloudFront, Elastic Beanstalk, and DynamoDB improved understanding of scalable and managed cloud infrastructure.

The project also demonstrated the importance of security, scalability, and performance optimization in modern web applications. Overall, the development process enhanced technical knowledge in full-stack development, cloud deployment, and system integration.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

The project successfully developed and deployed a cloud-based full-stack e-commerce web application using modern AWS cloud services and web technologies. The system provides functionalities such as secure user authentication, product browsing, cart management, and order placement through a scalable and reliable cloud-native architecture. By integrating services such as Amazon S3, CloudFront, Elastic Beanstalk, Cognito, and DynamoDB, the application achieved efficient performance, secure communication, and simplified infrastructure management.

The frontend application built using React delivered an interactive and responsive user interface, while the Node.js and Express backend handled API communication and business logic efficiently. AWS Cognito ensured secure authentication using JWT-based authorization, and DynamoDB provided scalable and low-latency data storage for order management.

The implementation of CI/CD using GitHub Actions automated the deployment workflow and improved development efficiency. Cloud-native features such as auto-scaling, load balancing, and CDN-based content delivery enhanced system availability, scalability, and performance.

Overall, the project demonstrated the effective use of cloud computing, DevOps practices, and modern web technologies to build a scalable and secure e-commerce platform. The system successfully met the project objectives related to scalability, security, performance, and cloud deployment.

6.2 FUTURE WORKS

Although the current system provides the core functionalities of a cloud-based e-commerce platform, several enhancements can be implemented in the future to improve functionality and user experience.

Advanced AI-based recommendation systems can be integrated to provide more accurate and personalized product suggestions based on user behavior and purchase history. Real-time analytics dashboards can also be implemented for vendors to monitor sales trends and customer activity more effectively.

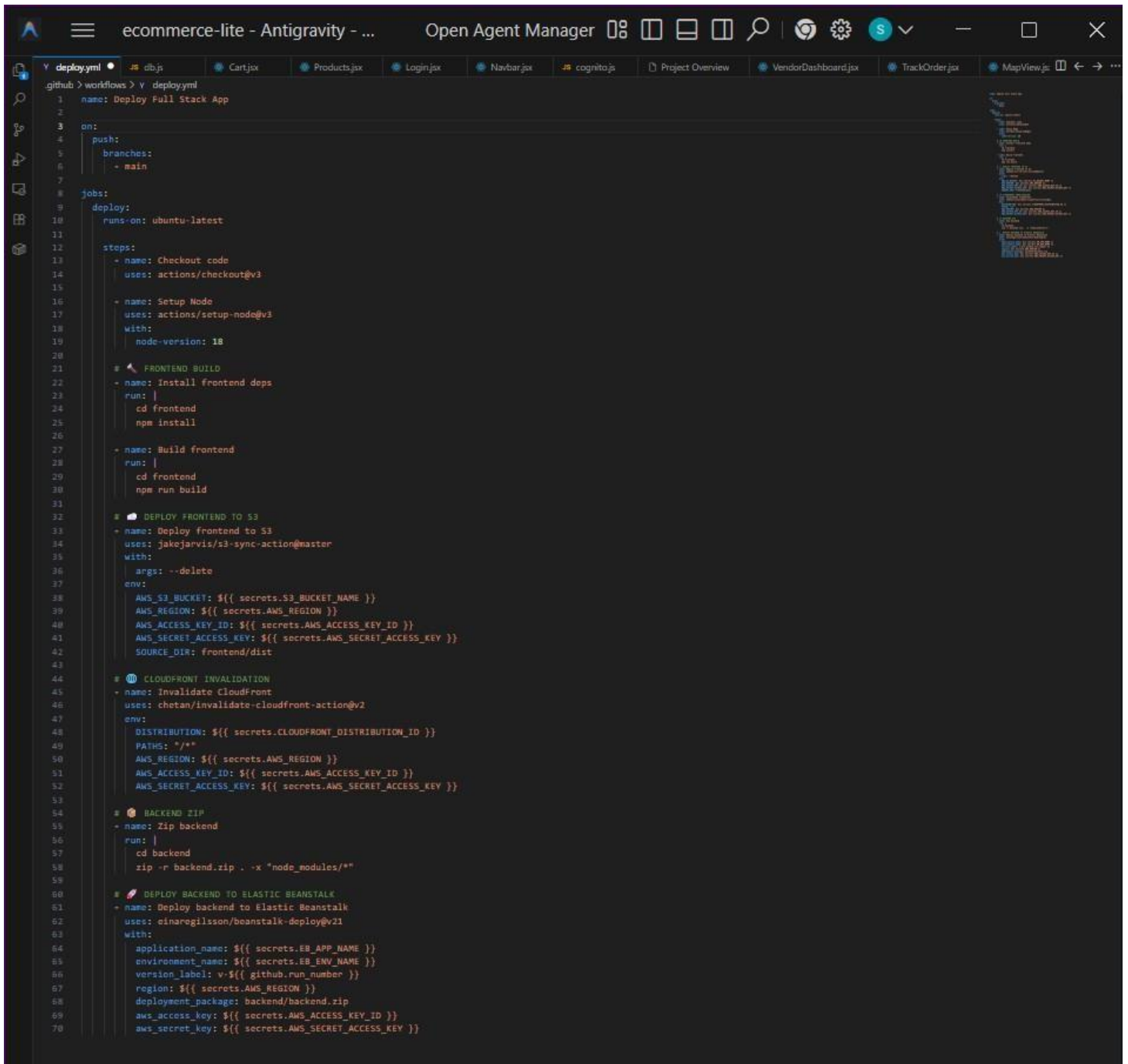
Payment gateway integration using services such as Stripe or Razorpay can be added to support secure online transactions. Additional features such as order tracking, email notifications, and real-time chat support can further enhance user interaction and system usability.

The system can also be extended into a mobile application using cross-platform frameworks such as React Native or Flutter. Container orchestration technologies such as Kubernetes can be integrated for advanced deployment management and scalability in large-scale production environments.

Future improvements may also include enhanced monitoring, AI-powered customer support, predictive inventory management, and multi-region cloud deployment for improved global availability and disaster recovery capabilities.

APPENDICES

Appendix A – GitHub workflow YAML screenshot



```
1 name: Deploy Full Stack App
2
3 on:
4   push:
5     branches:
6       - main
7
8 jobs:
9   deploy:
10    runs-on: ubuntu-latest
11
12    steps:
13      - name: Checkout code
14        uses: actions/checkout@v3
15
16      - name: Setup Node
17        uses: actions/setup-node@v3
18        with:
19          node-version: 18
20
21      # FRONTEND BUILD
22      - name: Install frontend deps
23        run: |
24          cd frontend
25          npm install
26
27      - name: Build frontend
28        run: |
29          cd frontend
30          npm run build
31
32      # DEPLOY FRONTEND TO S3
33      - name: Deploy frontend to S3
34        uses: jakejarvis/s3-sync-action@master
35        with:
36          args: --delete
37        env:
38          AWS_S3_BUCKET: ${ secrets.S3_BUCKET_NAME }
39          AWS_REGION: ${ secrets.AWS_REGION }
40          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
41          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
42          SOURCE_DIR: frontend/dist
43
44      # CLOUDFRONT INVALIDATION
45      - name: Invalidate cloudfront
46        uses: chetan/invalidate-cloudfront-action@v2
47        env:
48          DISTRIBUTION: ${ secrets.CLOUDFRONT_DISTRIBUTION_ID }
49          PATHS: /*
50          AWS_REGION: ${ secrets.AWS_REGION }
51          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
52          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
53
54      # BACKEND ZIP
55      - name: Zip backend
56        run: |
57          cd backend
58          zip -r backend.zip . -x "node_modules/*"
59
60      # DEPLOY BACKEND TO ELASTIC BEANSTALK
61      - name: Deploy backend to Elastic Beanstalk
62        uses: einaregilsson/beanstalk-deploy@v21
63        with:
64          application_name: ${ secrets.EB_APP_NAME }
65          environment_name: ${ secrets.EB_ENV_NAME }
66          version_label: v-${ github.run_number }
67          region: ${ secrets.AWS_REGION }
68          deployment_package: backend/backend.zip
69          aws_access_key: ${ secrets.AWS_ACCESS_KEY_ID }
70          aws_secret_key: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

Figure 9. GitHub Actions Workflow Configuration

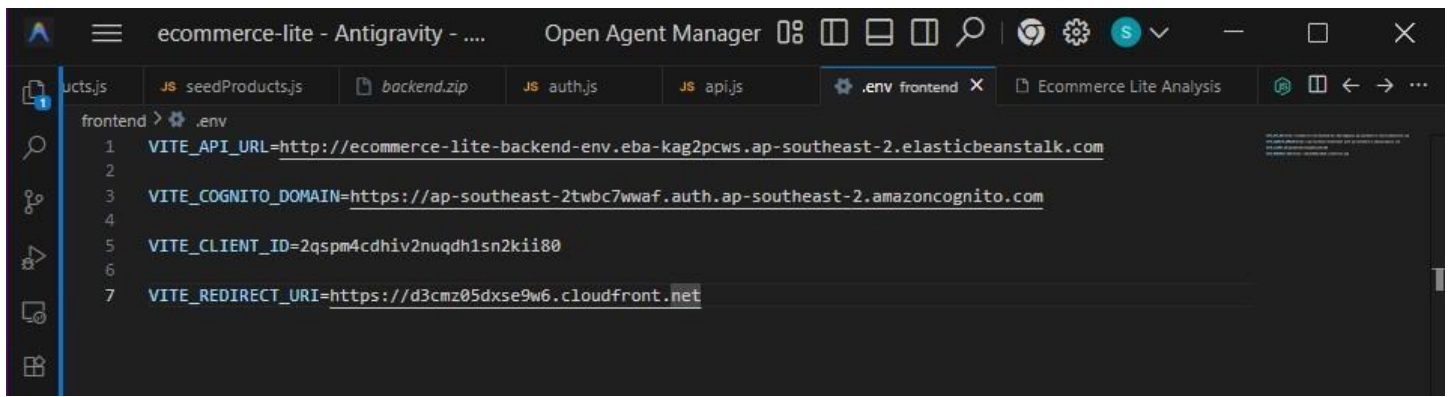


Figure 10. Application Environment Variables Setup

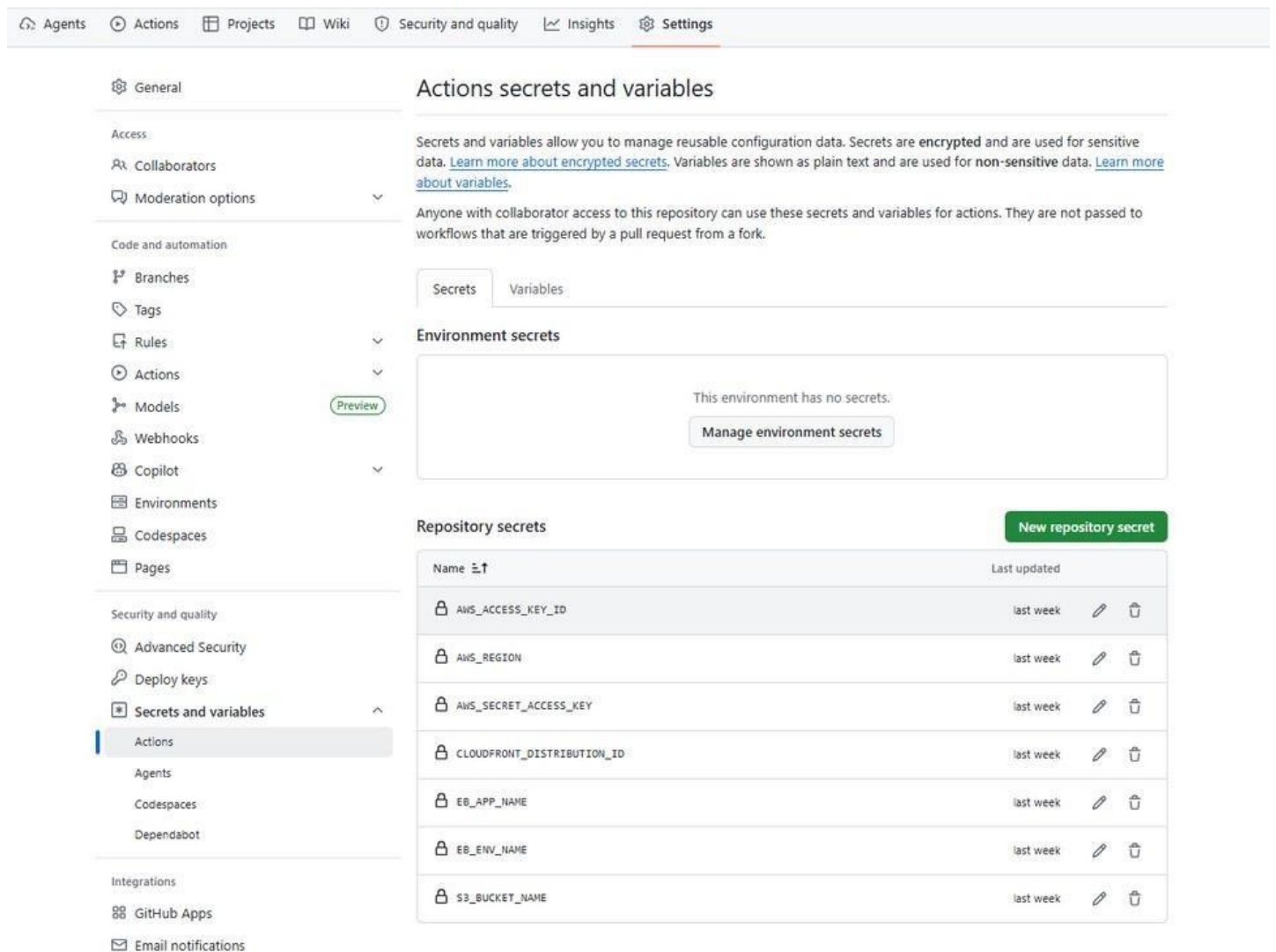


Figure 11. Deployment Environment Configuration

☰

Catriona20 / ecommerce-aws-app

🔍

👤

+

🕒

🔗

📄

📧

🌐

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Actions

New workflow

All workflows

Deploy Full Stack App

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

5 workflow runs

Workflow

Event

Status

Branch

Actor

✓ Fix auth and cart export

Deploy Full Stack App #11: Commit [48f1b35](#) pushed by [Catriona20](#)

main

May 8, 11:26 PM GMT+5:30

1m 15s

...

✓ Fix localCart export

Deploy Full Stack App #10: Commit [1b599a3](#) pushed by [Catriona20](#)

main

May 8, 11:21 PM GMT+5:30

1m 23s

...

✓ products connected to dynam...

Deploy Full Stack App #9: Commit [5c025b6](#) pushed by [Catriona20](#)

main

May 4, 12:30 PM GMT+5:30

1m 26s

...

✓ dynamodb access enabled

Deploy Full Stack App #8: Commit [72902b8](#) pushed by [Catriona20](#)

main

May 4, 12:14 PM GMT+5:30

1m 29s

...

✓ final fix EB deploy

Deploy Full Stack App #7: Commit [210dd21](#) pushed by [Catriona20](#)

main

May 4, 11:34 AM GMT+5:30

1m 23s

...

Figure 12. Successful CI/CD Deployment Execution

1.Archeiture Diagram

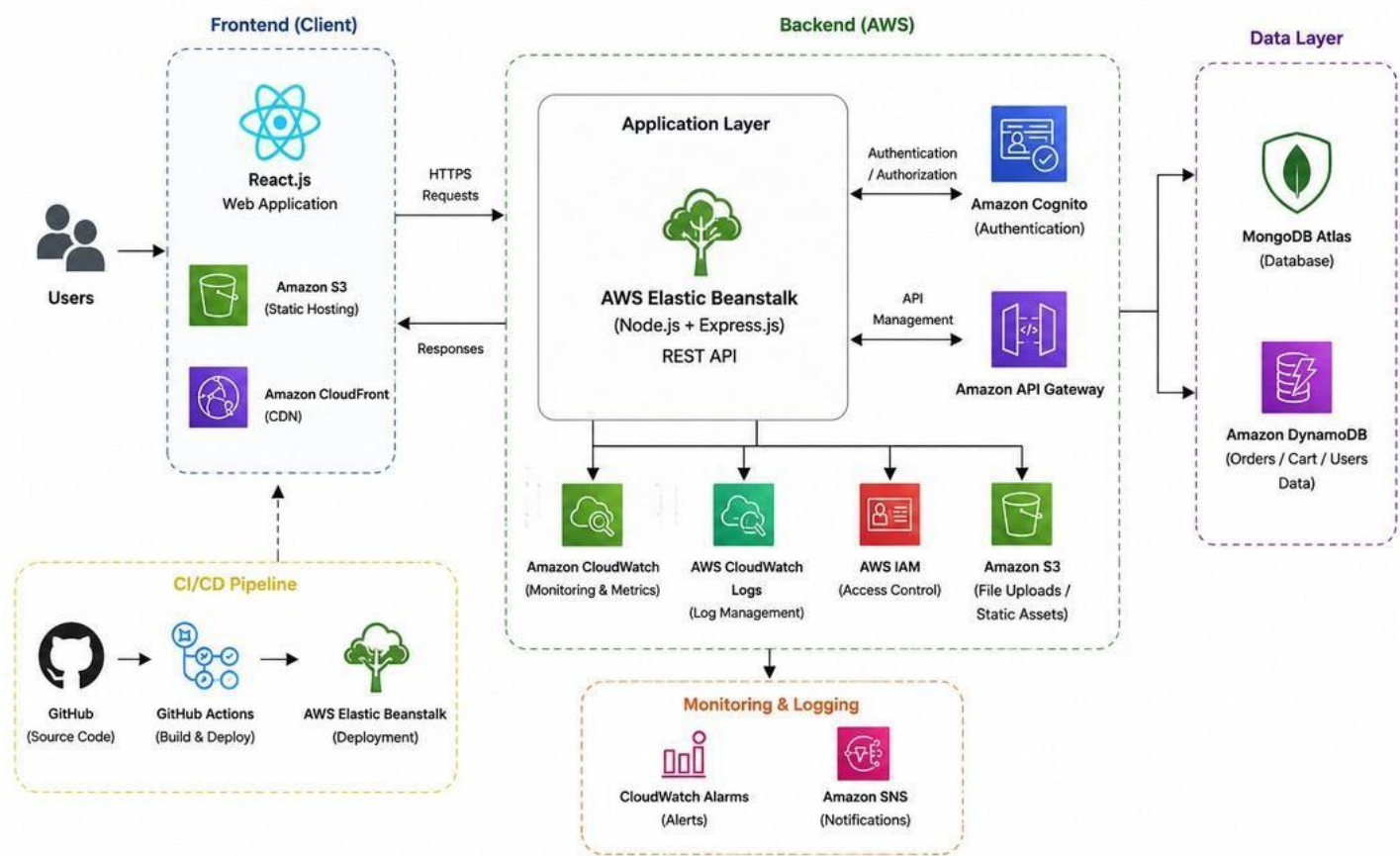


Figure 13. Workflow diagram

Surplus Lite – Cloud Architecture

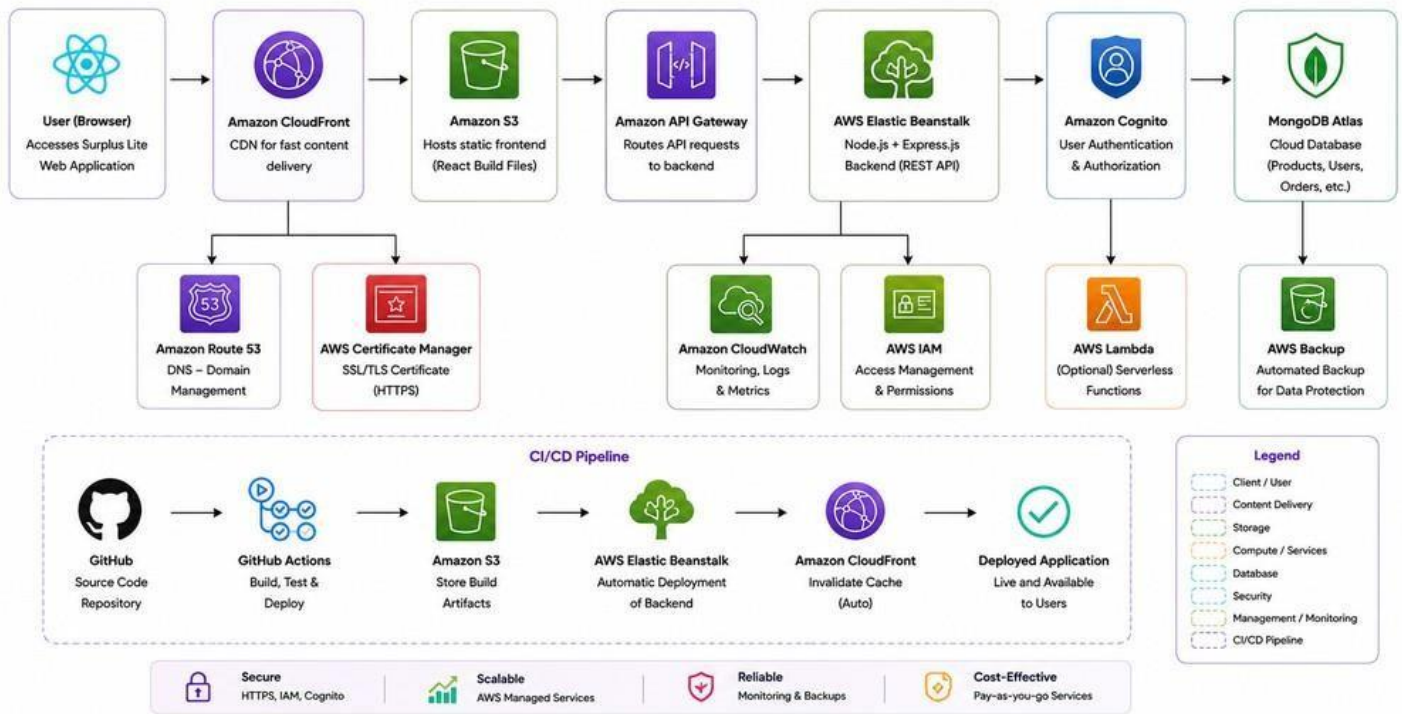


Figure 14. CI/CD Deployment Execution Logs

[Code](#)[Issues](#)[Pull requests](#)[Agents](#)[Actions](#)[Projects](#)[Wiki](#)[Security and quality](#)[Insights](#)[Settings](#)

← Deploy Full Stack App

✓ dynamodb access enabled #8

Re-run all jobs

...

Summary

All jobs

deploy

Run details

Usage

Workflow file

Annotations

1 warning

deploy

succeeded last week in 1m 25s

Search logs

↺ ⚙

> ✓ Set up job

3s

> ✓ Build jakejarvis/s3-sync-action@master

16s

1 ▶ Build container for action use: '/home/runner/work/_actions/jakejarvis/s3-sync-action/master/Dockerfile'.

> ✓ Checkout code

1s

> ✓ Setup Node

5s

> ✓ Install frontend deps

3s

> ✓ Build frontend

3s

> ✓ Deploy frontend to S3

8s

> ✓ Invalidate CloudFront

7s

> ✓ Zip backend

0s

> ✓ Deploy backend to Elastic Beanstalk

35s

> ✓ Post Setup Node

0s

> ✓ Post Checkout code

1s

> ✓ Complete job

0s

Figure 15. Link Scans